



Mark Scheme (Results)

Summer 2026

Pearson Edexcel International GCSE
In Computer Science (4CP0)
Paper 2B: Application of Computational Thinking

Edexcel and BTEC Qualifications

Edexcel and BTEC qualifications are awarded by Pearson, the UK's largest awarding body. We provide a wide range of qualifications including academic, vocational, occupational and specific programmes for employers. For further information visit our qualifications websites at www.edexcel.com or www.btec.co.uk. Alternatively, you can get in touch with us using the details on our contact us page at www.edexcel.com/contactus

Pearson: helping people progress, everywhere

Pearson aspires to be the world's leading learning company. Our aim is to help everyone progress in their lives through education. We believe in every kind of learning, for all kinds of people, wherever they are in the world. We've been involved in education for over 150 years, and by working across 70 countries, in 100 languages, we have built an international reputation for our commitment to high standards and raising achievement through innovation in education. Find out more about how we can help you and your students at:

www.pearson.com/uk

Summer 2026

Question Paper Log Number P83971A

Publication Code 4CPO_2B_2606_MS

All the material in this publication is copyright

© Pearson

General Marking Guidance

- All candidates must receive the same treatment. Examiners must mark the first candidate in exactly the same way as they mark the last.
- Mark schemes should be applied positively. Candidates must be rewarded for what they have shown they can do rather than penalised for omissions.
- Examiners should mark according to the mark scheme not according to their perception of where the grade boundaries may lie.
- There is no ceiling on achievement. All marks on the mark scheme should be used appropriately.
- All the marks on the mark scheme are designed to be awarded. Examiners should always award full marks if deserved, i.e. if the answer matches the mark scheme. Examiners should also be prepared to award zero marks if the candidate's response is not worthy of credit according to the mark scheme.
- Where some judgement is required, mark schemes will provide the principles by which marks will be awarded and exemplification may be limited.
- When examiners are in doubt regarding the application of the mark scheme to a candidate's response, the team leader must be consulted.
- Crossed out work should be marked UNLESS the candidate has replaced it with an alternative response.

Question	mp	Answer	Additional Guidance	Mark
1(a)(i)	A1	The only correct answer is C <i>A is not correct because it stores true/false</i> <i>B is not correct because it stores whole numbers</i> <i>D is not correct because it stores text</i>	The correct answer is: Real	1

Question	mp	Answer	Additional Guidance	Mark								
1(a)(ii)	A2 A3	Award one mark for each correct cell:	Accept reasonable alternative terms e.g. 'acceptable'	2								
		<table><tr><th>Type of test data</th><th>Test data</th></tr><tr><td>Boundary</td><td>1.5</td></tr><tr><td>Normal/Valid</td><td>2.0</td></tr><tr><td>Erroneous/Invalid</td><td>0.5</td></tr></table>			Type of test data	Test data	Boundary	1.5	Normal/Valid	2.0	Erroneous/Invalid	0.5
		Type of test data			Test data							
		Boundary			1.5							
		Normal/Valid			2.0							
Erroneous/Invalid	0.5											

Question	mp	Answer	Additional guidance	Mark
1(b)(i)	B1	Line 14		1
1(b)(ii)	B2	weight	Accept weight initialisation e.g. <code>int weight = 0;</code> do not accept line number	1
1(b)(iii)	B3	Award one mark for any of the following - get height - calculate bmi	do not accept line number ignore case and comment indicators	1
1(b)(iv)	B4	Line 14	accept range of lines e.g. lines 6 to 7 (must start with line that contains <code>while</code>)	1
1(b)(v)	B5	Line 24		1

Question	mp	Answer	Additional guidance	Mark
1(c)		Award one mark for each of:		
	C1	Correct spelling of ReadLine	C#: Line 25	
	C2	Change multiply to plus <code>minsWeek = minsWeek + minsToday</code>	C#: Line 26	
	C3	Change <code>if (level == "e" && minsWeek > 300)</code> to <code>if (level == "e" && minsWeek < 300)</code>	C#: Line 29 Accept <=	
	C4	Change <code>elif (level == "c" minsWeek < 200)</code> to <code>elif (level == "c" && minsWeek < 200)</code>	C#: Line 34 Ignore <=	
				4

Paper 2 – Application of Computational Thinking

```
1  // Q01c
2  using System;
3
4  namespace Q01c
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10              // Program variables
11              string[] days = { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday" };
12              int minsWeek = 0;
13              char level = 'e';
14              int minsToday = 0;
15
16              // Correct the following code
17
18              Console.WriteLine("Please enter e (elite) or c (club): ");
19              level = Console.ReadKey().KeyChar; // get level
20              Console.WriteLine();
21
22              for (int i = 0; i < 7; i++)
23              {
24                  Console.WriteLine("Please enter total minutes exercise on " + days[i] + ">");
25                  minsToday = int.Parse(Console.ReadLine());
26                  minsWeek = minsWeek + minsToday;
27              }
28
29              if (level == 'e' && minsWeek < 300) // check if elite runner has NOT completed enough exercise
30              {
31                  Console.WriteLine("Weekly exercise: " + minsWeek + " minutes");
32                  Console.WriteLine("You have not met your exercise target of 300 minutes.");
33              }
34              else if (level == 'c' && minsWeek < 200) // check if club runner has NOT completed enough exercise
35              {
36                  Console.WriteLine("Weekly exercise: " + minsWeek + " minutes");
37                  Console.WriteLine("You have not met your exercise target of 200 minutes.");
38              }
39              else
40              {
41                  Console.WriteLine("Weekly exercise: " + minsWeek + " minutes");
42                  Console.WriteLine("Well done, you have met your weekly exercise target.");
43              }
44          }
45      }
46  }
```

Question	mp	Answer	Additional Guidance	Mark
2(a)	A1	80: Just Right (1)	Ignore quote marks, capital letters and minor spelling errors	1
	A2	-2: End (1)		1
	A3	16: Low Evens (1)	ignore an additional attempt to receive new number (line 18)	1

Question	mp	Answer	Additional Guidance	Mark
2(b)	B1	The only correct answer is D <i>A is not correct because encryption is making data meaningless</i> <i>B is not correct because a function is a type of subprogram</i> <i>C is not correct because repetition is a type of loop</i>	The correct answer is Selection	1

Question	mp	Answer	Additional guidance	Mark
2(c)	C1 C2	Award up to two marks for a linked explanation such as: • The program will crash (1) because the wrong data type is entered (1) • A type error occurs (1) because this string cannot be used as an int (1) • A runtime error occurs (1) because program expected an integer (1) • Value error is given (1) as this string cannot be converted to an int (1)	Error on its own is not enough. Can mix and match as long as link makes sense.	2

Question	mp	Answer	Additional guidance	Mark
2(d)	D1 D2	Award up to two marks for a linked description such as: Divides (1) (one number by another) and returns the remainder (1)	allow answers relating to Figure 2 e.g: Divides number by 2 (1) and outputs 1 or 0 / determines if number is odd or even (1)	2

Question	mp	Answer	Additional guidance	Mark
2(e)		Award one mark for each of:		4
	E1	Add <code>int.Parse(Console.ReadLine())</code> to height and radius (1)	award mark for either height or radius input (don't need both for this mark) any suitable method e.g. <code>Convert.ToInt32(Console.ReadLine())</code> etc..	
	E2	Correct volume formula <code>PI * radius * radius * height</code> <code>Math.Pow(radius, 2) * height * Math.PI</code>	Award any correct formula using given variables, in any order	
	E3	Use of constant <code>PI</code> in the formula	Must use exact constant name Allow consistent change of constant name	
	E4	<code>Console.WriteLine("Cylinder Volume: " + volume + " cm3")</code>	Must include <code>volume</code> and any suitable string Suitable string includes "volume" or "cm3"	

```
1  // Q02e
2  using System;
3
4  namespace Q02e
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10              // constant PI taken from math library
11              const double PI = Math.PI;
12
13              // program variables
14              int radius = 0;
15              int height = 0;
16              double volume = 0.0;
17
18              // complete the code to get integer values for radius and height
19              Console.Write("Enter cylinder radius in cm: ");
20              radius = int.Parse(Console.ReadLine());
21              Console.Write("Enter cylinder height in cm: ");
22              height = int.Parse(Console.ReadLine());
23
24              // complete the code to calculate the volume of the cylinder
25              volume = (PI * (radius * radius)) * height;
26
27              // rounding volume to whole number before output message
28              volume = Math.Round(volume);
29
30              // add a line of code to output volume within a suitable message
31              Console.WriteLine("Cylinder Volume: " + volume + "cm3");
32              Console.ReadLine();
33          }
34      }
35  }
```

Question	mp	Answer	Additional guidance	Mark
3(a)		Award one mark for each of:		
	A1	deposit and period taken as input	must contain appropriate prompt	
	A2	deposit and period as correct data types (deposit: double, and period: int)	Allow FT from A1 if period and year taken as input	
	A3	While loop used and terminates correctly using period this mark requires both: while (period > 0){ (condition) and period - 1 (decrement)	Allow FT from A2 (period not int type)	
	A4	Pseudocode line 17 formula coded correctly 17 SET value TO value + value * INTEREST	value = value + value * INTEREST value = value * INTEREST + value value = value * (1 + INTEREST) value = (1 + INTEREST) * value value += value * INTEREST	
	A5	Counter implemented for year		
	A6	Appropriate output as indicated for each year	year values must be correct (1, 2, 3 etc..) Allow infinite loop where year is incrementing and displayed Any value for savings – must be present Message should appear as given in pseudocode – Allow missing currency do not accept hard coded	
	A7	All correct output	Allow other currencies for MP6 and MP7 Allow use of round to get int for earned	7

Test data for examiners

deposit	period	earned (output)
2500	5	Please enter the amount of money you wish to invest: £2500 Please enter the number of years you wish to invest for: 5 Year 1 Savings value: £ 2612 Year 2 Savings value: £ 2730 Year 3 Savings value: £ 2852 Year 4 Savings value: £ 2981 Year 5 Savings value: £ 3115 Total earned: £ 615
1000	10	please enter the amount of money you wish to invest: £1000 Please enter the number of years you wish to invest for: 10 Year 1 Savings value: £ 1045 Year 2 Savings value: £ 1092 Year 3 Savings value: £ 1141 Year 4 Savings value: £ 1192 Year 5 Savings value: £ 1246 Year 6 Savings value: £ 1302 Year 7 Savings value: £ 1360 Year 8 Savings value: £ 1422 Year 9 Savings value: £ 1486 Year 10 Savings value: £ 1552 Total earned: £ 552

```

1  // Q03a
2
3  using System;
4
5  public class Q03a
6  {
7      public static void Main(string[] args)
8      {
9          // program variables
10         double deposit = 0.0;
11         double value = 0.0;
12         double earned = 0.0;
13         int period = 1;
14         int year = 0;
15
16         const double INTEREST = 0.045; // 4.5% fixed annual interest rate
17
18         // write your code below this line
19         // -----
20         Console.Write("Please enter the amount of money you wish to invest: £");
21         deposit = double.Parse(Console.ReadLine());
22
23         Console.Write("Please enter the number of years you wish to invest for: ");
24         period = int.Parse(Console.ReadLine());
25
26         value = deposit;
27
28         while (period > 0)
29         {
30             value = value + value * INTEREST;
31             year = year + 1;
32             period = period - 1;
33             Console.WriteLine("Year " + year + " Investment value: £" + Math.Round(value));
34         }
35
36         earned = value - deposit;
37         Console.WriteLine("Total earned: £" + Math.Round(earned));
38     }
39 }

```

Question	mp	Answer	Additional Guidance	Mark
3(b)(i)	B1 B2	Award one mark for any of the following up to a maximum of two marks: • Immutable/value cannot be changed (while running) (1) • Declared once / declared at beginning (1) • Value set by programmer / set at compile time (1) • Unique identifier (1) • Has a data type (1)	'named location in RAM' acceptable for BP4 Accept name in UPPERCASE (convention rather than characteristic)	2

Question	mp	Answer	Additional guidance	Mark
3(b)(ii)	B1 B2	Award up to two marks for a linked description such as: • Global variables are accessible from anywhere in a program (1), whereas local variables can only be accessed from within the scope/subprogram/subroutine in which they are declared (1) • Local variables are declared within subprograms/subroutines (1), whereas global variables are declared in the main program / outside of subprograms/subroutines (1) • Local variables are created when a subroutine/subprogram starts and are deleted when it ends (1), whereas global variables are in RAM for the entire running/execution of the program (1)		2

Question	mp	Answer	Additional Guidance	Mark
4(a)	A1	The only correct answer is C <i>A is not correct because a library is a collection of subprograms</i> <i>B is not correct because a parameter is a local variable</i> <i>D is not correct because a string stores a sequence of characters</i>	The correct answer is record	1

Question	mp	Answer	Additional guidance	Mark
4(b)	B1 B2	Award one mark for data structure: • Array/list (1) Award one mark for any of the following Justifications: • The values are all the same data type (1) • Multiple values are stored (under a single variable name) (1) • Because a loop can iterate across the values in the array (1) • Because individual values can be indexed (1) • Array size is known in advance (max 31)		2

Question	mp	Answer	Additional guidance	Mark
4(c)	C1 C2	Award up to two marks for a linked explanation such as: • It is easier to see what the code is intended to do (1) because the implementation is hidden (1) • There is less code to read through (1) because lots of the code is contained in the subprograms / separate files (libraries) (1) • What the code is doing is clear / easier to understand (1) because subprograms names reflect their purpose (1) • The amount of code (in main) is reduced (1) as unnecessary/repetitive code is removed (1)		2

Question	mp	Answer	Additional guidance	Mark
4(d)		Award one mark for each of:		
	D1	Display last 2 numbers to user	Syntactically correct output statement with "string" and 2 int variables Not hard coded	
	D2	Get int input from user and Output suitable message for each correctly entered value e.g. correct	<code>Var = int.Parse(Console.ReadLine())</code>	
	D3	Accurate comparison of user input to calculated value. Must be syntactically correct if/while statement e.g. <code>if (userInput == nextNumber){ while (userInput == nextNumber){</code>	Ignore incorrect calculation of nextNumber. userInput must be the variable taken for D2	
	D4	Calculate next number in sequence and accurately update sequence values	lastNumber and secondLastNumber must hold correct values throughout. Candidate may rename or have own version of these variables	
	D5	End program only when incorrect number entered – must follow Fibonacci sequence	Accept while True / break Do not award this mark as FT	
	D6	Program output matches Fibonacci sequence for: 1,2,3,5.. 9 to exit loop Include suitable final message and accurate number of values added (4 correct values added)	including appropriate messages	6

```

1  // Q04d
2
3  using System;
4
5  namespace Q04d
6  {
7      class Q04d
8      {
9          static void Main(string[] args)
10         {
11             // program variables
12             int lastNumber = 1;
13             int secondLastNumber = 0;
14             int countCorrect = 0;
15             int userInput = 0;
16
17             // write your code below this line
18             // -----
19
20             // loop until incorrect sequence number entered
21             bool endProgram = false;
22             while (!endProgram)
23             {
24                 // output last two numbers with a suitable message
25                 Console.WriteLine("Last two numbers in the sequence are " + secondLastNumber + " and " + lastNumber);
26
27                 // calculate next number
28                 int nextFibonacci = lastNumber + secondLastNumber;
29
30                 // get user input
31                 Console.Write("Enter the next number in the sequence: ");
32                 userInput = int.Parse(Console.ReadLine());
33
34                 // check if user has input correct number and take appropriate steps
35                 if (userInput == nextFibonacci)
36                 {
37                     Console.WriteLine("Correct \n");
38                     secondLastNumber = lastNumber;
39                     lastNumber = nextFibonacci;
40                     countCorrect = countCorrect + 1;
41                 }
42                 // deal with incorrect user input
43                 else
44                 {
45                     endProgram = true;
46                 }
47             }
48
49             // tell user how many numbers they added to the fibonacci array
50             Console.WriteLine("Incorrect. Numbers you added to sequence: " + countCorrect);
51         }
52     }
53 }
54

```

Question	mp	Answer	Additional guidance	Mark
5(a)	A1	7 / seven (1)		1

Question	mp	Answer			Additional guidance	Mark
5(b)	B1 B2 B3 B4		Number	Order compared	All FT on 1 st value being in 7 th row only. 2 FT marks for this response	
			2328			
			3369			
			3928	2		
			4091	2		
			5017	4		
			5909	3		
			6266	1		
			6499	1		
			7148			
			7789			
			8027			
			8412			
			9027			
			9031			
	9256					

4

Question	mp	Answer	Additional guidance	Mark
5(c)				
	C1	Counter controlled accurately	Both; increment and reset after each run	
	C2	Method for checking if run has ended and only then adding LN to string e.g. - if (<string>[i] == <string>[i-1]){ - if (char_current == char_previous){	Any method Must work e.g. Correct IF and <string> += LN inside ELSE Correct While and <string> += LN outside loop	
	C3	Loop to process all items in instruction string	Loop that targets string/string length	
	C4	Counter cast to string/char	Any method Must target an int that candidate reasonably expects is a run count	
	C5	Construct an accurate compressed string	Ignore missed E3 at end, missing N3 at start or missing N1 in middle Allow swapping order of N and L Do not award - N11W19S13 - N1N2N3W4W5....E49	
	C6	Construct fully accurate compressed string	Including N3 at start, N1 in middle and E3 at end	
	C7	Open (for writing) and close file	Constant FILENAME not necessary for this mark Accept any file handling methods e.g. using (StreamWriter writer = new StreamWriter(FILENAME))	
	C8	String written to file	Even if compressed string not correct If required, examiner can add line to close text file to award this mark	
	C9	Appropriate use of FILENAME constant and final message output		
				9

```

1  // Q05c
2
3  using System.IO;
4
5  public class Q05c
6  {
7      public static void Main(string[] args)
8      {
9          // program variables
10         string instructionString = "NNNNWWSSSSSEEEENWWWWWWWWSSSSSWWWSSSSWNNNNNNNEEE";
11         string compressedString = "";
12         int count = 0;
13         char previousChar = instructionString[0];
14         string FILENAME = "instructions.txt";
15
16         // -----
17         // write your code below this line
18
19         // iterate over the string
20         foreach (char thisChar in instructionString)
21         {
22             // count number of occurrences for each direction
23             if (thisChar == previousChar)
24             {
25                 count = count + 1;
26             }
27             // update compressed string
28             else
29             {
30                 compressedString = compressedString + previousChar + count.ToString();
31                 count = 1;
32                 // update previous char at end of each run
33                 previousChar = thisChar;
34             }
35         }
36
37         // deal with final character
38         compressedString = compressedString + previousChar + count.ToString();
39
40         // write string to file
41         string filePath = "C:\\\" + FILENAME;
42         StreamWriter writer = new StreamWriter(filePath);
43         writer.Write(compressedString);
44         writer.Close();
45
46         // output confirmation message
47         Console.WriteLine("Instruction file has been created");
48     }
49 }

```

Question	mp	Answer	Additional guidance	Mark
6		Award one mark for each of:		
	A1	Integer input taken for player choice of category	Does not have to be validated	
	A2	Selection used to compare category values		
	A3	Correct use of 2D indexing	Must get at least 1 inner item	
	A4	Appropriate winner/lose/draw string displayed to console		
	A5	Mechanism for accurately tracking player/computer scores. Must include all 3: • <code>playerScore += 1</code> (when player wins) • <code>computerScore = computerScore + 1</code> (when cpu wins) • draw has no increment (when both elements are ==)	must be within selection statement	
	A6	User input validated accurately. Must work in all rounds	does not need additional error message	
	A7	Loop controlled by half length of the deck array or length of accurately generated player/cpu array	do not award hard coded must play 5 rounds do not award full deck array unless step of 2	
	A8	Mechanism for dealing unique cards to player and computer and no card repeated	e.g. alternate cards or first half and second half etc.	
	A9	Player card accurately displayed each round	must be within loop and appear at least once	
	A10	Accurate tracking and display of (win/lose/draw) scores		
	A11	All five rounds played with all correct output including computer cards and final scores	Ignore missed validation for this mark	
				11

LBMS (AOs: 3xAO3.2a, 3xAO3.2b, 3xAO3.3c) (Tgt: 3, 3, 3); Spec: 1.1.6(2), 1.2.1(2), 1.2.2(2), 2.1.1, 2.1.2, 2.1.5

PBMS: 2.1.1, 2.3.2(2), 2.3.3, 2.4.1, 2.5.1(2), 2.6.1(3), 2.6.3

Award up to a maximum of nine marks using the levels-based mark scheme below.				Mark
Band 0	Band 1 (1-3 marks)	Band 2 (4-6 marks)	Band 3 (7-9 marks)	
No rewardable content	Little attempt to decompose the problem into component parts	Some attempt to decompose the problem into component parts	The problem has been decomposed into component parts	(9)
	Some parts of the logic are clear and appropriate to the problem	Most parts of the logic are clear and mostly appropriate to the problem	The logic is clear and appropriate to the problem	
	Some appropriate use and manipulation of data types, variables, data structures and program constructs	The use and manipulation of data types, variables and data structures and program constructs is mostly appropriate	The use and manipulation of data types, variables and data structures and program constructs is appropriate	
	Parts of the code are clear and readable	Code is mostly clear and readable	Code is clear and readable	
	Finished program will not be flexible enough with other data sets or input	Finished program will function with some but not all other data sets or input	Finished program could be used with other data sets or input	
	The program meets some of the given requirements	The program meets most of the given requirements	The program fully meets the given requirements	

Paper 2 – Application of Computational Thinking

```
1  // Q06
2
3  using System;
4
5  public class Q06
6  {
7
8      static void Main(string[] args)
9      {
10         // program variables
11         int[,] pack = new int[,]
12         {
13             { 3, 4, 8 }, { 4, 2, 16 }, { 3, 6, 16 }, { 2, 10, 8 },
14             { 3, 2, 16 }, { 2, 8, 16 }, { 4, 4, 4 }, { 4, 4, 32 },
15             { 4, 6, 16 }, { 3, 4, 8 }
16         };
17
18         // -----
19         // Write your code below this line
20         string[] categories = new string[] { "Storage", "Cores", "RAM" };
21
22         int[,] playerCards = new int[pack.GetLength(0) / 2 + (pack.GetLength(0) % 2 == 0 ? 0 : 1), pack.GetLength(1)];
23         int[,] computerCards = new int[pack.GetLength(0) / 2, pack.GetLength(1)];
24
25
26         int playerScore = 0;
27         int computerScore = 0;
28
29         int playerSelection = 0;
30
31         // deal cards
32         for (int i = 0, j = 0, k = 0; i < pack.GetLength(0); i++)
33         {
34             if (i % 2 == 0)
35             {
36                 for (int l = 0; l < pack.GetLength(1); l++)
37                 {
38                     playerCards[j, l] = pack[i, l];
39                 }
40                 j++;
41             }
42             else
43             {
44                 for (int l = 0; l < pack.GetLength(1); l++)
45                 {
46                     computerCards[k, l] = pack[i, l];
47                 }
48                 k++;
49             }
50         }
51
52         // play game
53         for (int card = 0; card < playerCards.GetLength(0); card++)
54         {
55             Console.WriteLine("\nYour card:");
56             for (int category = 0; category < 3; category++)
57             {
58                 Console.WriteLine(categories[category] + ": " + playerCards[card, category]);
59             }
60             playerSelection = 0;
61
62             // get player choice of category
63             while (playerSelection < 1 || playerSelection > 3) // allow only valid input 1-3
64             {
65                 Console.WriteLine("Please enter 1 for Storage, 2 for Cores or 3 for RAM\n>");
66                 playerSelection = int.Parse(Console.ReadLine());
67             }
68             playerSelection = playerSelection - 1;
```

Paper 2 – Application of Computational Thinking

```
69
70     // show computer card
71     Console.WriteLine("\nComputer card:");
72     for (int category = 0; category < 3; category++)
73     {
74         Console.WriteLine(categories[category] + ": " + computerCards[card, category]);
75     }
76
77     // decide who wins card
78     if (playerCards[card, playerSelection] == computerCards[card, playerSelection])
79     {
80         Console.WriteLine("\nDraw \n");
81     }
82     else if (playerCards[card, playerSelection] > computerCards[card, playerSelection])
83     {
84         Console.WriteLine("\nYou win!\n");
85         playerScore = playerScore + 1;
86     }
87     else
88     {
89         Console.WriteLine("\nYou lose!\n");
90         computerScore = computerScore + 1;
91     }
92 }
93
94 // print final Score
95 Console.WriteLine("Final score \nPlayer " + playerScore + " - " + computerScore + " Computer");
96 }
97 }
```